

1. Project Overview

This project demonstrates how to build a reusable Flutter Chat UI module, wrap it inside a native Android (Kotlin) SDK library, and publish it automatically using GitHub Actions to a GitHub Pages-hosted Maven repository. Any native Android app can then consume this SDK by adding a single Gradle dependency - no Flutter installation required on the consumer's side.

Repository Structure

```
flutter_chat_sdk/  
+-- flutter_chat_module/      # Flutter module (chat UI)  
+-- native-android-sdk/      # Kotlin library wrapping Flutter  
+-- KotlinDemoApp/           # Demo app consuming the SDK  
+-- .github/workflows/        # CI/CD pipeline  
+-- test_flutter_sdk_in_flutter/ # Flutter-to-Flutter test app
```

High-Level Flow

- Flutter module provides the chat UI (screens, widgets, models).
- Native Android SDK wraps the Flutter module and exposes a simple Kotlin API (FlutterChatSDK).
- GitHub Actions CI/CD builds the Flutter AAR, packages the Kotlin SDK, and publishes to GitHub Pages Maven.
- Consumer apps add the Maven URL and dependency - zero Flutter setup required.

Technology Stack

- Flutter 3.x (stable channel), Dart SDK >=3.0.0
- Kotlin 1.9.22, AGP 8.3.0, Gradle 8.6
- Java 17 source/target compatibility
- minSdk 24, compileSdk/targetSdk 34
- GitHub Actions + GitHub Pages (public Maven repo)

2. Building the Flutter Module

Step 1: Create a Flutter Module

A Flutter module (as opposed to a regular app or plugin) is specifically designed for embedding into existing native apps. It is created with 'flutter create --template module' and produces a .android/ directory with Gradle build files that can generate an AAR.

pubspec.yaml Configuration

The pubspec.yaml declares the module metadata. The key section is the 'flutter > module' block which sets the Android package name and iOS bundle identifier:

```
name: flutter_chat_module
version: 1.0.0
environment:
  sdk: '>=3.0.0 <4.0.0'
flutter:
  module:
    androidPackage: com.flutterchat.module
    iosBundleIdentifier: com.flutterchat.module
    uses-material-design: true
```

Step 2: Code Structure

The module follows clean architecture with models, services, screens, and widgets:

- models/chat_message.dart - ChatMessage data class with JSON serialization
- models/chat_user.dart - ChatUser data class (id, name, email)
- services/platform_channel.dart - MethodChannel bridge to native Kotlin
- screens/chat_screen.dart - Main chat UI with message list and input
- widgets/chat_bubble.dart - Individual message bubble widget
- widgets/message_input.dart - Text input bar with send button

Step 3: Platform Channel (Flutter <-> Kotlin Bridge)

The PlatformChannel class creates a MethodChannel named 'com.flutterchat.sdk/bridge'. When Flutter is ready, it sends a 'flutterReady' signal to Kotlin. Kotlin responds with 'initData' containing user info (JSON). Flutter can send events back via 'nativeEvent' and 'rnEvent' methods. This bidirectional communication is the core of the SDK.

```
// Channel definition
static const MethodChannel _channel =
    MethodChannel('com.flutterchat.sdk/bridge');

// Flutter signals readiness
_channel.invokeMethod('flutterReady', null);

// Kotlin sends user data -> Flutter handles it
case 'initData':
    final user = ChatUser.fromJson(data);
    _onUserDataReceived?.call(user);
```

Step 4: Build the Flutter AAR

The Flutter module is compiled into an AAR (Android Archive) using the command below. This produces a local Maven repository under build/host/outputs/repo/ containing the flutter_release AAR and all transitive Flutter engine dependencies.

```
cd flutter_chat_module
flutter build aar --no-profile --no-debug --build-number 1
```

3. Native Android SDK (Kotlin Wrapper)

The native-android-sdk project is a standard Android library that depends on the Flutter module AAR and exposes a clean, simple Kotlin API. Consumers never interact with Flutter directly.

Key Classes

ChatUser.kt - A simple data class mirroring the Flutter side:

```
data class ChatUser(
    val id: String,
    val name: String,
    val email: String
)
```

FlutterChatCallback.kt - Interface for receiving events from Flutter:

```
interface FlutterChatCallback {
    fun onNativeEvent(eventName: String, data: String)
    fun onRnEvent(eventName: String, data: String)
    fun onError(error: String) {}
}
```

FlutterChatSDK.kt - The public singleton entry point (2 methods):

```
object FlutterChatSDK {
    fun setCallback(callback: FlutterChatCallback) { ... }
    fun openChat(context: Context, user: ChatUser) {
        pendingUser = user
        val intent = Intent(context, FlutterChatActivity::class.java)
        context.startActivity(intent)
    }
}
```

FlutterChatActivity.kt - Hosts the Flutter engine and handles MethodChannel communication. When Flutter sends 'flutterReady', the Activity responds with 'initData' containing user JSON. When Flutter sends 'nativeEvent' or 'rnEvent', the Activity forwards them to the callback.

SDK build.gradle (Maven Publishing)

The SDK's build.gradle applies 'maven-publish' plugin. It depends on the Flutter module AAR ('com.flutterchat.module:flutter_release') and pins the Flutter engine version to the exact build hash to avoid resolution failures on Google's Maven. CI passes version parameters:

```
gradle :sdk:publish \
    -PMAVEN_REPO_URL="file://$PWD/build/maven-repo" \
    -PSDK_VERSION=1.0.4 \
    -PFLUTTER_ENGINE_VERSION="1.0.0-<hash>" \
    -PFLUTTER_MODULE_VERSION="1.0.0"
```

AndroidManifest.xml

The SDK declares FlutterChatActivity in its own manifest. Since it's bundled in the AAR, consumers don't need to add any activity declarations:

```
<activity
    android:name="com.flutterchat.sdk.FlutterChatActivity"
    android:exported="false"
    android:theme="@style/Theme.AppCompat.Light.NoActionBar"
    android:hardwareAccelerated="true"
    android:windowSoftInputMode="adjustResize" />
```

4. GitHub Actions CI/CD & GitHub Pages Publishing

The entire build-and-publish pipeline is automated via a single GitHub Actions workflow file: `.github/workflows/publish-native-sdk.yml`. It is triggered manually (`workflow_dispatch`) and accepts a version number as input.

Workflow Steps (Step-by-Step)

1. Checkout Repository

Uses `actions/checkout@v4` to clone the repo.

2. Set up Java 17

Uses `actions/setup-java@v4` with Temurin distribution.

3. Set up Flutter

Uses `subosito/flutter-action@v2` on the stable channel.

4. Strip Version Prefix

Removes any leading 'v' from the input version (e.g., `v1.0.4` -> `1.0.4`).

5. Build Flutter AAR

Runs `'flutter build aar --no-profile --no-debug'` in `flutter_chat_module/`. This generates the AAR and a local Maven repo at `build/host/outputs/repo/`.

6. Get Flutter Engine Version

Extracts the exact engine revision from `'flutter --version --machine'`. This hash is critical for pinning transitive `io.flutter` dependencies.

7. Copy Flutter Maven Repo

Copies the Flutter AAR repo into `native-android-sdk/flutter-repo/` so the Kotlin SDK build can resolve the Flutter module dependency locally.

8. Detect Flutter Module Version

Reads the actual directory name from the Flutter build output to get the published version string (matches `pubspec.yaml` version).

9. Build & Publish SDK

Runs `'gradle :sdk:publish'` with Maven URL, SDK version, engine version, and module version passed as `-P` parameters. Output goes to `build/maven-repo/`.

10. Deploy to GitHub Pages

Checks out or creates the `gh-pages` branch. Copies all Maven artifacts (Flutter module + native SDK) into a `maven/` directory. Commits and force-pushes.

11. Create GitHub Release

Uses `softprops/action-gh-release@v2` to create a tagged release with installation instructions in the release notes.

GitHub Pages as Maven Repository

GitHub Pages serves the `gh-pages` branch as a static website. Since Maven repositories are just a directory structure of POM and AAR/JAR files, GitHub Pages works perfectly as a free, public Maven host. The URL pattern is:

`https://<username>.github.io/<repo>/maven`

No authentication is required for consumers. Each publish overwrites the `gh-pages` branch with all accumulated versions.

5. Versioning & Pushing Code to GitHub

How Versioning Works

The SDK uses a manual versioning approach via the GitHub Actions `workflow_dispatch` input. When you trigger the 'Publish Native SDK' workflow, you provide a version string (e.g., `1.0.4`). This version flows through the pipeline as follows:

- The input version becomes the `SDK_VERSION` Gradle property, which sets the Maven artifact version (`com.flutterchat:chat-sdk:<VERSION>`).
- The Flutter module version comes from `pubspec.yaml` (currently 1.0.0) and is auto-detected from the build output.
- The Flutter engine version is extracted at build time from `'flutter --version --machine'` and pinned in the POM.
- A Git tag (`v<VERSION>`) and GitHub Release are created automatically.
- Consumers update their dependency: implementation `'com.flutterchat:chat-sdk:1.0.4'`

Pushing Code (Development Workflow)

1. Make changes to Flutter module code or Kotlin SDK code locally.
2. Commit and push to the main branch:

```
git add . && git commit -m "feat: add feature X" && git push origin main
```
3. Go to GitHub > Actions > 'Publish Native SDK' > Run workflow.
4. Enter the new version number (e.g., 1.0.5) and click 'Run workflow'.
5. The pipeline builds, publishes to GitHub Pages, and creates a release.
6. Consumers update their version in `build.gradle` to use the new version.

Version History Example

```
v1.0.0 - Initial release
v1.0.1 - Bug fixes
v1.0.2 - UI improvements
v1.0.3 - Added RN event support
v1.0.4 - Current latest
```

6. Consuming the SDK in a Native Kotlin App

The `KotlinDemoApp` in the repository demonstrates how a native Android app consumes the SDK. Here are all the steps a consumer needs to follow:

Step 1: Add Maven Repositories (root `build.gradle`)

Add the GitHub Pages Maven URL and Flutter engine repository:

```
allprojects {
    repositories {
        google()
        mavenCentral()
        maven { url "https://neerajj911.github.io/
            new_flutter_chat_sdk/maven" }
        maven { url "https://storage.googleapis.com/
            download.flutter.io" }
    }
}
```

Step 2: Add SDK Dependency (app `build.gradle`)

```
dependencies {
    implementation 'com.flutterchat:chat-sdk:1.0.4'
}
```

Step 3: Set Up Callback (`MainActivity.kt`)

Register a callback to receive events from the Flutter chat module:

```
FlutterChatSDK.setCallback(object : FlutterChatCallback {
    override fun onNativeEvent(
        eventName: String, data: String) {
        Toast.makeText(this@MainActivity,
            "Event: $eventName", Toast.LENGTH_SHORT).show()
    }
})
```

```

    }
    override fun onRnEvent(
        eventName: String, data: String) {}
    override fun onError(error: String) {
        Toast.makeText(this@MainActivity,
            error, Toast.LENGTH_SHORT).show()
    }
}
})

```

Step 4: Open Chat Screen

Create a ChatUser and call openChat - the SDK handles everything else:

```

val user = ChatUser(
    id = email,
    name = name,
    email = email
)
FlutterChatSDK.openChat(this, user)

```

What Happens Internally

- FlutterChatSDK.openChat() stores the user and launches FlutterChatActivity.
- FlutterChatActivity starts the Flutter engine and sets up the MethodChannel.
- Flutter sends 'flutterReady' -> Kotlin responds with 'initData' (user JSON).
- Flutter parses the user data and shows the chat screen with a welcome message.
- User sends a message -> Flutter sends 'nativeEvent' -> Kotlin callback fires.

Consumer Requirements

- minSdk 24 or higher
- Java 17 source/target compatibility
- Internet permission (for Flutter engine to load)
- No Flutter SDK installation needed!

7. Summary & Architecture Diagram

The complete flow can be summarized as:

[Flutter Module] --(AAR)--> [Kotlin SDK] --(Maven/GitHub Pages)--> [Consumer App]

Key architectural decisions:

- Flutter module pattern allows embedding Flutter UI in native apps without requiring Flutter on the consumer side.
- MethodChannel provides type-safe bidirectional communication between Dart and Kotlin.
- GitHub Pages as a Maven repo eliminates the need for Sonatype/Maven Central setup.
- Manual workflow_dispatch versioning gives full control over releases.
- Engine version pinning prevents Gradle resolution failures on Google's storage.
- SDK's AndroidManifest auto-merges into the consumer app - zero configuration needed.